

REXX, TSO, and Batch

Writing REXX scripts and running TSO in batch mode





WHAT IS REXX?

```

opt_json=0
numeric digits 12
parse arg opts
parse var opts opts '--' paths
paths=strip(paths)
if wordpos('-H',translate(opts))>0 then signal help
/* if not an omvs environment, run in an OMVS environment
parse source . . . . . rxenv .
if rxenv<>'OMVS' then
do
if syscalls('ON')>4 then
do
call say 'Cannot initialize as a z/OS UNIX process'
if rxenv='AXR' then /* if sysrexex tell op what is r
call say 'Logon to the console with userid that d
'z/OS UNIX'

exit 1
end
tmp='/bin/zlsof'
rv=bpxwunix('_BPX_SHAREAS=NO' tmp arg(1),,out.,err.)
/* print command output */
do i=1 to out.0
call say out.i
end
do i=1 to err.0
call say err.i

```

REXX is the main scripting language on z/OS. With native support for Python now included in z/OS, it's possible the use of REXX will decline in future, but there is so much REXX code already in place and so many common jobstreams and system configuration procedures using REXX that it will surely be around for some time to come.

The acronym stands for Restructured Extended Executor. The REXX language was developed at IBM by Mike Cowlshaw in 1979, and was subsequently open sourced and ported to numerous platforms.

Although REXX runs on many platforms, it is a *multi-platform* tool and not a *cross-platform* tool. That is, the implementations are platform-specific and the source code is not portable. When working on z/OS, stick to the IBM documentation.

WHAT IS REXX?

REXX for z/OS can run either interpreted or compiled. It expects to run in a TSO environment; in fact, the IBM documentation for REXX is a chapter inside the documentation for TSO. But it can be wrangled into running outside of TSO.

REXX is used in many configuration and initialization processes, including z/OS IPL processes.

A REXX script or program begins with a comment line that tells the interpreter it's looking at REXX source. The script ends with an EXIT statement.

A script containing just those two lines will run without errors and produce no output.

Between those statements we can write REXX code to do useful work. The script HOWDY uses the REXX statement SAY to write to the standard output device. In the TSO environment, that's the 3270 terminal.

```
VIEW      IBMUSER.LAB.REXX(ALPHOMEG) - 01.01      Columns 00001 00072
Command ==>                                     Scroll ==> CSR
***** ***** Top of Data *****
000100 /* REXX */
000300 exit 0
***** ***** Bottom of Data *****
```

```
READY
ex 'ibmuser.lab.rexx(alphomeg)'
READY
```

```
EDIT      IBMUSER.LAB.REXX(HOWDY) - 01.00      Columns 00001 00072
Command ==>                                     Scroll ==> CSR
***** ***** Top of Data *****
000100 /* REXX */
000200 say "Hello, World!"
000300 exit 0
***** ***** Bottom of Data *****
```

```
READY
ex 'ibmuser.lab.rexx(howdy)'
Hello, World!
READY
```

WHAT IS REXX?

```

EDIT      IBMUSER.LAB.REXX(REXLIST) - 01.07      Columns 00001 00072
Command ==>                                     Scroll ==> CSR
***** ***** Top of Data *****
000100 /* REXX */
000200 ADDRESS TSO
000210 "LISTDS 'IBMUSER.*.JCL'"
000300 exit 0
***** ***** Bottom of Data *****

```

Provided the script is running in a TSO environment – and on z/OS it normally will be – it can issue TSO commands by telling the interpreter what it intends to do, with the ADDRESS TSO statement, and then giving a TSO command enclosed in quotes.

```

READY
ex 'ibmuser.lab.rexx(rexlist)'
IBMUSER.DEV.JCL
--RECFM=LRECL-BLKSIZE=DSORG
FB 80 27920 PO
--VOLUMES--
USRVS1
IBMUSER.GAMA01.JCL
--RECFM=LRECL-BLKSIZE=DSORG
FB 80 27920 PO
--VOLUMES--
USRVS1
IBMUSER.HELLO.JCL
--RECFM=LRECL-BLKSIZE=DSORG
FB 80 27920 PO
--VOLUMES--
USRVS1
IBMUSER.LAB.JCL
--RECFM=LRECL-BLKSIZE=DSORG
FB 80 27920 PO
--VOLUMES--
USRVS1
IBMUSER.TST.JCL
--RECFM=LRECL-BLKSIZE=DSORG
FB 80 27920 PO
--VOLUMES--
USRVS1
READY

```

WHAT IS REXX?

REXX has many features of a complete programming language. In that sense, using it feels something like using PowerShell. Here's an example that takes an argument and calculates a factorial using recursion.

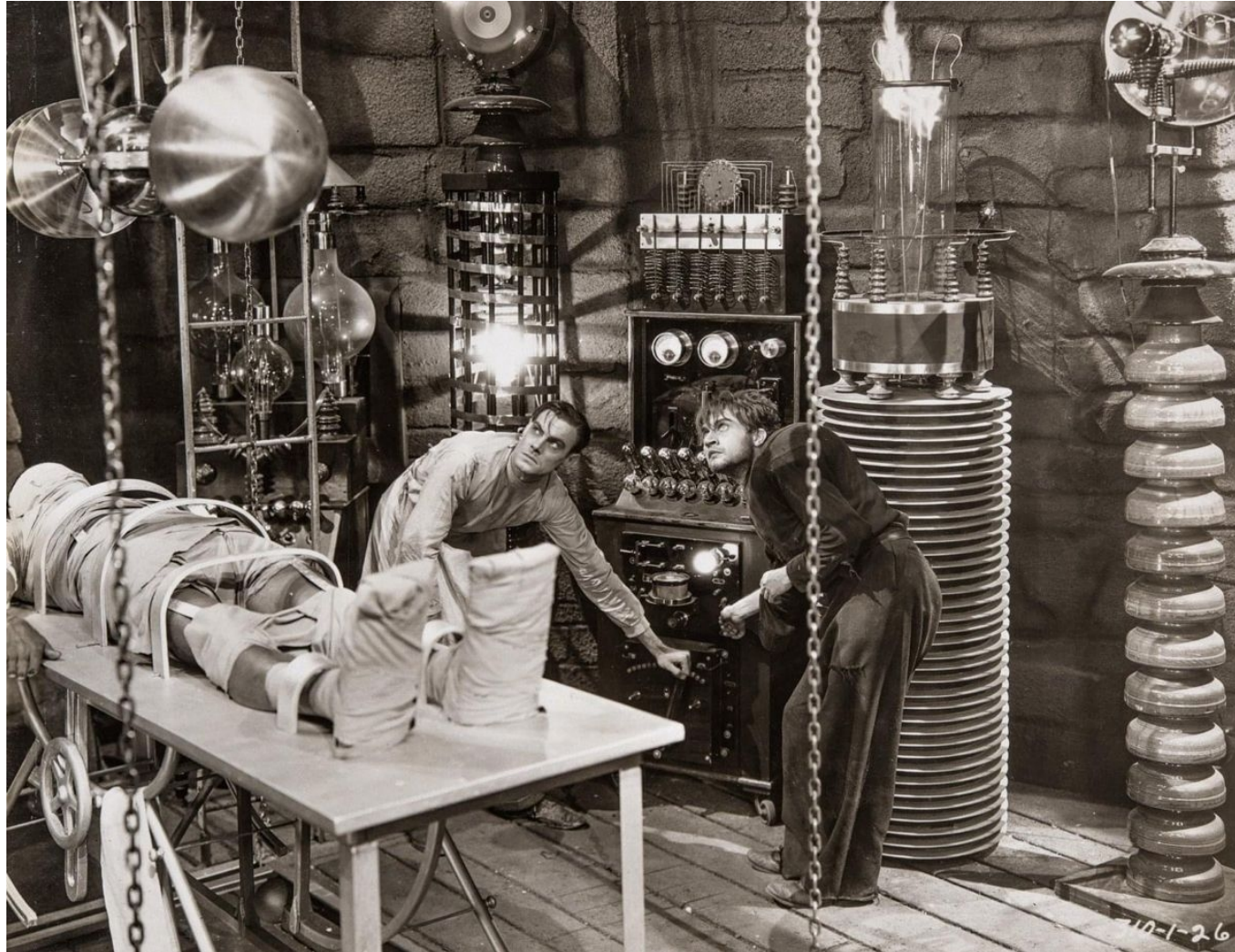
This sort of thing is more than a novelty, as we sometimes need to process data sets whose qualifiers match a pattern, and dive into multiple levels of qualifiers.

You can also see from this example that REXX source doesn't have to be in upper case. It's case-insensitive.

```
EDIT      IBMUSER.LAB.REXX(FACTORIA) - 01.00      Column
Command ==>                                         Sc
***** ***** Top of Data *****
000100 /* REXX */
000200 /* Recursive internal function execution... */
000210 arg x
000220 say x'!' = factorial(x)
000230 exit 0
000300 factorial: procedure /* Calculate factorial by */
000400 arg n                /* recursive invocation. */
000500 if n=0 then return 1
000600 return factorial(n-1) * n
***** ***** Bottom of Data *****
```

```
READY
ex 'ibmuser.lab.rexx(factoria)' '5'
5! = 120
READY
█
```


LAB Z/OS 20: WRITING A REXX SCRIPT



RUNNING TSO IN BATCH MODE

TSO is the "command line" of MVS, although it doesn't look much like a Unix/Linux command line. You can execute TSO in a batch job through JCL, and run TSO commands in the batch environment.

The batch TSO program is IKJEFT01.

TSO thinks it's talking to a 3270 terminal. In batch mode, TSO reads "terminal input" from the data set with DDNAME SYSTSIN, and writes "terminal output" to the data set with DDNAME SYSTSPRT.

The sample job at right specifies SYSTSIN as an inline data set containing a TSO LISTDS command. It directs SYSTSPRT to SYSOUT, so the output will appear in a spooler data set viewable with SDSF.

The diagram illustrates the execution of a batch TSO job. It consists of three main components:

- JCL Editor View (Left):** Shows the JCL for a batch job named 'IBMUUSER.LAB.JCL(TSOBATCH)'. Key lines include:
 - 000001 //IBMUUSER JOB , 'TSO BATCH',
 - 000002 // CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
 - 000003 // NOTIFY=&SYSUID
 - 000004 //STEP1 EXEC PGM=IKJEFT01 DYNAMNBR=75,TIME=1440
 - 000005 //SYSPRINT DD SYSOUT=*
 - 000006 //SYSTSPRT DD SYSOUT=*
 - 000007 //SYSTEM DD DUMMY
 - 000008 //SYSTSIN DD *
 - 000009 LISTDS 'IBMUUSER.*.JCL'
 - 000010 /*
- SDSF JOB DATA SET DISPLAY (Top Right):** A table showing job details for IBMUSER (JOB02097). The table has columns: NP, DDNAME, StepName, ProcStep, DSID, Owner, C, Dest, Rec-Cnt, and Page. The row for SYSTSPRT is highlighted with a red box.

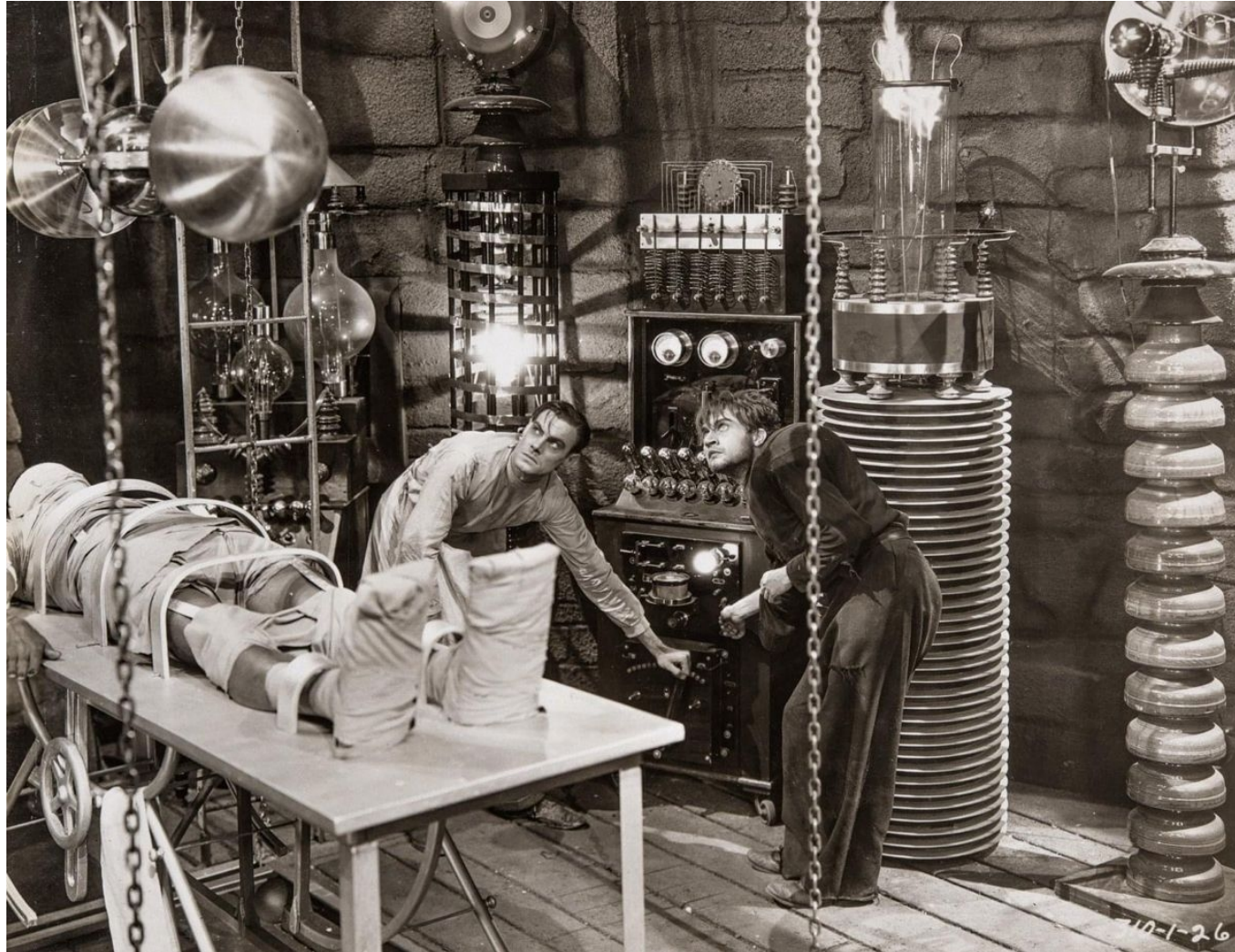
NP	DDNAME	StepName	ProcStep	DSID	Owner	C	Dest	Rec-Cnt	Page
2	JESMSG LG	JES2		2	IBMUUSER	X	LOCAL	16	
3	JESJCL	JES2		3	IBMUUSER	X	LOCAL	9	
4	JESYSMSG	JES2		4	IBMUUSER	X	LOCAL	31	
103	SYSTSPRT	STEP1		103	IBMUUSER	X	LOCAL	29	
- SDSF OUTPUT DISPLAY (Bottom Right):** Shows the output of the TSO command. The command 'LISTDS 'IBMUUSER.*.JCL'' is entered, and the output shows the details of the dataset 'IBMUUSER.DEV.JCL', including its record format (RECFM=LRECL), block size (BLKSIZE=DSORG), and volume (VOLUME=USRVS1).


```

SDSF OUTPUT DISPLAY IBMUSER JOB02097 DSID 103 LINE 0
COMMAND INPUT ==>
***** TOP OF DATA *****
READY
LISTDS 'IBMUUSER.*.JCL'
IBMUUSER.DEV.JCL
--RECFM=LRECL-BLKSIZE=DSORG
FB 80 27920 PO
--VOLUME=
USRVS1
      
```

Arrows indicate the flow of data: from the JCL editor to the SDSF JOB DATA SET DISPLAY, and from the SDSF JOB DATA SET DISPLAY to the SDSF OUTPUT DISPLAY.

LAB Z/OS 21: RUNNING REXX FROM TSO BATCH



WHAT YOU KNOW AND WHAT YOU CAN DO

- You can write a simple REXX script.
- You can run a REXX script from the TSO READY prompt.
- You can run a REXX script using TSO batch.

WHAT'S NEXT

- USS, shell scripts, and batch